

# Two Pointers Revision

19 problems

Fast recall cards for the current two pointers package. Read the problem name, picture the pointer movement, then check the sample and bug magnets before opening the Java file. Exported in expanded card mode.

## #01 CellsWithOddValues

Easy

Row/column parity Easy [Source](#) [LeetCode](#)

**Before reading: can row and column parity replace repeated full-matrix updates?**

### PROBLEM DESCRIPTION

Start with a zero matrix. For every index pair, increment its whole row and whole column, then count cells whose final value is odd.

### INPUT

$m = 2, n = 3$   
indices =  $[[0,1], [1,1]]$

### OUTPUT

6

### TIME

 $O(k + m * n)$ 

### SPACE

 $O(m + n)$ 

### DATA STRUCTURES

Row parity array, column parity array

### WHEN TO RECOGNIZE IT

When full rows and columns receive repeated updates, but you only care whether each final value is odd or even.

### VISUAL HOOK

r0:+1

r1:+1

c0:0

c1:+2

c2:0

- 1 Toggle the chosen row state.
- 2 Toggle the chosen column state.
- 3 A cell is odd when row and column parity differ.

Track row and column parity instead of touching the whole matrix after every operation. A cell is odd when its row parity and column parity differ.

### CORE MOVE

Toggle or count the affected row and column for every operation. At the end, combine each row state with each column state and count odd cells.

### CODE SKELETON

```
rows = int[m]; cols = int[n]
for [r,c] in indices: rows[r]++; cols[c]++
odd = 0
for r, c:
    if (rows[r] + cols[c]) % 2 == 1: odd++
```

**BRUTE FORCE / BASELINE**

Start with the simplest correct approach that directly follows the problem statement, then compare it with the optimized move above.

**ALTERNATE APPROACHES**

- No strong alternate approach needed for quick revision; focus on the core move.

**BUG MAGNETS**

- The current Java method is incomplete and always returns 0.
- You do not need to build and repeatedly update the full matrix.

## #02 PairWithTargetSum

Easy

Opposite-end sum Easy [Source](#) [LeetCode](#)

**Before reading: which pointer move makes the current sum closer to target?**

**PROBLEM DESCRIPTION**

Given a sorted array and a target, return the indices of two numbers whose sum equals the target.

**INPUT**

arr = [1, 2, 3, 4, 6]  
target = 6

**OUTPUT**

[1, 3]

**TIME**

$O(n)$

**SPACE**

$O(1)$

**DATA STRUCTURES**

Left and right pointers

**WHEN TO RECOGNIZE IT**

When a sorted array asks for a pair with an exact sum and moving one endpoint changes the sum predictably.

**VISUAL HOOK**

- 1 Start at the smallest and largest values.
- 2 Too small: move left rightward.
- 3 Too large: move right leftward.

The smallest and largest values face each other. Too large means shrink from the right; too small means grow from the left.

**CORE MOVE**

Start at both ends. Compare `arr[left] + arr[right]` with the target, then move exactly the pointer that can correct the sum.

**CODE SKELETON**

```
left = 0; right = n - 1
while left < right:
    sum = arr[left] + arr[right]
    if sum == target: return [left, right]
    if sum < target: left++
    else: right--
```

**BRUTE FORCE / BASELINE**

Start with the simplest correct approach that directly follows the problem statement, then compare it with the optimized move above.

**ALTERNATE APPROACHES**

- No strong alternate approach needed for quick revision; focus on the core move.

**BUG MAGNETS**

- This two-pointer direction relies on the array being sorted.
- Return the index format expected by the caller; LeetCode Two Sum II expects one-based indexes.

## #03 RemoveDuplicates

Easy

Unique write slot Easy [Source](#) [LeetCode](#)

**Before reading: where should the next distinct value be written?**

**PROBLEM DESCRIPTION**

Remove duplicates from a sorted array in place and return how many unique values remain at the front.

**INPUT**

`arr = [2, 3, 3, 3, 6, 9, 9]`

**OUTPUT**

4  
`front = [2, 3, 6, 9]`

**TIME**

**$O(n)$**

**SPACE**

**$O(1)$**

**DATA STRUCTURES**

**Read pointer, last-unique pointer**

**WHEN TO RECOGNIZE IT**

When a sorted array must be compacted in place while preserving one copy of every distinct value.

**VISUAL HOOK**

- 1 Keep the latest unique value at the write boundary.
- 2 Scan each sorted value.
- 3 Copy only when the value changes.

The read pointer explores; the write boundary advances only when a genuinely new value appears.

**CORE MOVE**

Keep the last unique value at the write boundary. Scan forward and copy a value only when it differs from the last unique value.

**CODE SKELETON**

```
if arr.length == 0: return 0
write = 1
for read from 1:
    if arr[read] != arr[write-1]:
        arr[write++] = arr[read]
return write
```

**BRUTE FORCE / BASELINE**

Start with the simplest correct approach that directly follows the problem statement, then compare it with the optimized move above.

**ALTERNATE APPROACHES**

- No strong alternate approach needed for quick revision; focus on the core move.

**BUG MAGNETS**

- The current implementation returns 1 for an empty array; guard empty input first.
- The values after the returned unique length do not matter.

## #04 SquaringSortedArray

Easy

Largest square first Easy [Source](#) [LeetCode](#)

**Before reading: why must the next largest square come from an endpoint?**

**PROBLEM DESCRIPTION**

Square every value in a sorted array and return the squares in sorted order.

**INPUT**

arr = [-2, -1, 0, 2, 3]

**OUTPUT**

[0, 1, 4, 4, 9]

**TIME**  
 **$O(n)$** **SPACE**  
 **$O(n)$** **DATA STRUCTURES**  
**Two pointers, output array****WHEN TO RECOGNIZE IT**

When negatives break the original sorted order after a transform based on absolute value.

**VISUAL HOOK**

- 1 Compare absolute values at both ends.
- 2 Write the larger square into the last open slot.
- 3 Move that endpoint inward.

The largest square must come from one of the two ends. Fill the output from right to left.

**CORE MOVE**

Compare absolute values at both ends. Square the larger one into the last open output slot, then move that endpoint inward.

**CODE SKELETON**

```
left = 0; right = n - 1; write = n - 1
while left <= right:
    if abs(arr[left]) >= abs(arr[right]): use
        left++
    else: use right--
    squares[write--] = chosen * chosen
```

**BRUTE FORCE / BASELINE**

Start with the simplest correct approach that directly follows the problem statement, then compare it with the optimized move above.

**ALTERNATE APPROACHES**

- No strong alternate approach needed for quick revision; focus on the core move.

**BUG MAGNETS**

- Fill the result from the end, because you discover largest squares first.
- Compare absolute values before squaring.

## #05 ThreeSum

**Medium**Anchor plus pair Medium [Source](#) [LeetCode](#)**Before reading: how do you prevent the same triplet from being emitted twice?****PROBLEM DESCRIPTION**

Return all unique triplets whose values add up to zero.

**INPUT**

```
arr = [-3, 0, 1, 2, -1, 1, -2]
```

**OUTPUT**

```
[[-3,1,2], [-2,0,2], [-2,1,1], [-1,0,1]]
```

**TIME**

**$O(n^2)$**

**SPACE**

**$O(\log n)$  + output**

**DATA STRUCTURES**

**Sorted array, two pointers, result list**

**WHEN TO RECOGNIZE IT**

When you need unique triples and sorting lets each fixed first value become a two-sum search.

**VISUAL HOOK**

- 1 Sort and choose one anchor.
- 2 Use left/right to find the balancing pair.
- 3 Skip duplicate anchors and duplicate pair values.

Freeze one anchor. The remaining two pointers squeeze toward the pair that balances it to zero.

**CORE MOVE**

Sort first. For each non-duplicate anchor, run a left/right two-sum search and skip duplicates after recording a match.

**CODE SKELETON**

```

sort(arr)
for i:
    if i > 0 && arr[i] == arr[i-1]: continue
    left = i+1; right = n-1
    while left < right:
        move by sum; record zero and skip
        duplicates
  
```

**BRUTE FORCE / BASELINE**

Start with the simplest correct approach that directly follows the problem statement, then compare it with the optimized move above.

**ALTERNATE APPROACHES**

- No strong alternate approach needed for quick revision; focus on the core move.

**BUG MAGNETS**

- Skip duplicate anchors before starting the inner search.
- After recording a triplet, skip duplicate left/right values and still move both pointers once.

## #06 ThreeSumClosest

Medium

Closest anchored sum Medium [Source](#) [LeetCode](#)

**Before reading: when must the closest-so-far answer be updated?**

**PROBLEM DESCRIPTION**

Return the sum of three integers whose total is closest to the target.

**INPUT**

arr = [-3, -1, 1, 2]  
target = 1

**OUTPUT**

0

**TIME**

$O(n^2)$

**SPACE**

$O(\log n)$

**DATA STRUCTURES**

Sorted array, two pointers,  
best-so-far sum

**WHEN TO RECOGNIZE IT**

When exact equality is optional and every candidate triplet must update one closest-so-far answer.

**VISUAL HOOK**

- 1 Sort and fix an anchor.
- 2 Measure every current triplet's distance to target.
- 3 Update best before moving toward target.

Every pointer position creates a candidate gap to the target. Keep the smallest gap, then move toward the target.

**CORE MOVE**

Sort, fix an anchor, and scan with two pointers. Update the closest sum before moving right for a high sum or left for a low sum.

**CODE SKELETON**

```
sort(arr); best = first triplet sum
for i:
    left = i+1; right = n-1
    while left < right:
        sum = arr[i]+arr[left]+arr[right]
        update best by abs(target-sum)
        if sum < target: left++ else: right--
```

**BRUTE FORCE / BASELINE**

Start with the simplest correct approach that directly follows the problem statement, then compare it with the optimized move above.

**ALTERNATE APPROACHES**

- No strong alternate approach needed for quick revision; focus on the core move.

**BUG MAGNETS**

- Update the best answer in both pointer-movement branches.
- When two sums are equally close, this implementation keeps the smaller sum.

# #07 ThreeSumSmaller

Medium

Count a valid range Medium [Source](#) [LeetCode](#)

**Before reading: why does one valid sum prove right-left triplets are valid?**

## PROBLEM DESCRIPTION

Count how many index triplets have a sum strictly smaller than the target.

### INPUT

```
arr = [-2, 0, 1, 3]
target = 2
```

### OUTPUT

2

### TIME

$O(n^2)$

### SPACE

$O(\log n)$

### DATA STRUCTURES

Sorted array, two pointers, counter

### WHEN TO RECOGNIZE IT

When sorted order lets one valid endpoint prove that an entire range of smaller endpoints is also valid.

### VISUAL HOOK

anchor

L

...

R

valid pairs

=

R - L

- 1 Sort and fix an anchor.
- 2 Test the largest pair at left/right.
- 3 If valid, count the entire right-left range.

If anchor + left + right is already below target, every value between left and right also works with that same anchor and left.

### CORE MOVE

Sort and fix an anchor. If the largest current pair still gives a small sum, add right-left at once and move left; otherwise move right.

### CODE SKELETON

```
sort(arr); count = 0
for i:
    left = i+1; right = n-1
    while left < right:
        if sum < target: count += right-left;
        left++
        else: right--
```

### BRUTE FORCE / BASELINE

Start with the simplest correct approach that directly follows the problem statement, then compare it with the optimized move above.

### ALTERNATE APPROACHES

- No strong alternate approach needed for quick revision; focus on the core move.



**BUG MAGNETS**

- Count right-left, not just one valid triplet.
- The condition is strictly smaller, so a sum equal to target must move right.

## #08 DutchNationalFlagProblem

**Medium**Three-way partition Medium [Source](#) [LeetCode](#)**Before reading: which swapped-in value must be examined again?****PROBLEM DESCRIPTION**

Sort an array containing only 0, 1, and 2 in one pass and in place.

**INPUT**

arr = [2, 2, 0, 1, 2, 0]

**OUTPUT**

[0, 0, 1, 2, 2, 2]

**TIME****O(n)****SPACE****O(1)****DATA STRUCTURES****Low, scan, and high pointers****WHEN TO RECOGNIZE IT**

When values belong to exactly three groups and the array should be partitioned in one pass.

**VISUAL HOOK****0 zone****unknown zone****2 zone****L****i ->****<- R**

- 1 Zero goes to the low boundary.
- 2 One stays and scan advances.
- 3 Two goes to high; re-check the new scan value.

Maintain three zones: confirmed zeroes, unknown values, and confirmed twos. Ones naturally remain in the middle.

**CORE MOVE**

Swap zeroes toward low, twos toward high, and advance over ones. Re-examine the value swapped in from the high side.

**CODE SKELETON**

```
low = 0; scan = 0; high = n - 1
while scan <= high:
    if arr[scan] == 0: swap(low++, scan++)
    else if arr[scan] == 1: scan++
    else: swap(scan, high--)
```

**BRUTE FORCE / BASELINE**

Start with the simplest correct approach that directly follows the problem statement, then compare it with the optimized move above.

**ALTERNATE APPROACHES**

- No strong alternate approach needed for quick revision; focus on the core move.

**BUG MAGNETS**

- After moving a 2 right, do not skip the unknown value swapped into the scan position.
- The clean invariant stops when scan passes high, not when they merely meet.

## #09 BackspaceStringCompare

Easy

Reverse scan with skip counts Easy [Source](#) [LeetCode](#)**Before reading: what does a # mean when scanning from right to left?****PROBLEM DESCRIPTION**

Given two strings containing lowercase letters and '#', compare whether they are equal after each '#' deletes the previous available character. Do it without building the final strings.

**INPUT**

```
s = "bxj##tw"  
t = "bxo#j##tw"
```

**OUTPUT**

```
true
```

**TIME** **$O(n + m)$** **SPACE** **$O(1)$** **DATA STRUCTURES****Two reverse pointers, skip counters****WHEN TO RECOGNIZE IT**

When backspaces only affect earlier characters and equality can be decided by comparing the next live character from the end.

**VISUAL HOOK****s: b x j # # t w****live: b t w****t: b x o # j # # t w****live: b t w**

- 1 Move each pointer left until it lands on a live character.
- 2 A # increases skip count; a normal character is skipped if skip count is positive.
- 3 Compare the two live characters, then continue left.

Scan from the right as if each # creates a skip ticket. Spend skip tickets on letters until the next live character is visible.

**CORE MOVE**

Start both pointers at the end. For each string, move left past backspaces and skipped letters until a live character or -1 appears. Compare live characters, then move both pointers left.

**CODE SKELETON**

```
i = s.length - 1; j = t.length - 1
while i >= 0 or j >= 0:
    i = nextLiveIndex(s, i)
    j = nextLiveIndex(t, j)
    if i < 0 or j < 0: return i == j
    if s[i] != t[j]: return false
    i--; j--
return true
```

**BRUTE FORCE / BASELINE**

Build both final strings with a stack or StringBuilder, then compare them. It is straightforward, but it spends  $O(n + m)$  extra space.

**ALTERNATE APPROACHES**

- Use a stack for each string when clarity matters more than constant extra space.
- Use a helper that returns the next valid index for each string; this keeps the comparison loop small.

**BUG MAGNETS**

- Compare only after both pointers are moved to live characters.
- If one pointer runs out before the other, the strings differ.
- Multiple consecutive backspaces accumulate skip count.

## #10 ValidPalindromell

Easy

Spend one deletion Easy [Source](#) [LeetCode](#)

**Before reading: where do you spend the only deletion token?**

**PROBLEM DESCRIPTION**

Given a string, return true if it is already a palindrome or can become one after deleting at most one character.

**INPUT**

s = "abca"

**OUTPUT**

true

**TIME**

$O(n)$

**SPACE**

$O(1)$

**DATA STRUCTURES**

Two pointers, bounded  
palindrome check

**WHEN TO RECOGNIZE IT**

When the string must be almost symmetric and exactly one bad character is allowed to disappear.

**VISUAL HOOK**

a b c a

match

mismatch

try skip b or c

- 1 Move left and right inward while characters match.
- 2 At the first mismatch, try skipping the left character.
- 3 Also try skipping the right character; either successful inner check is enough.

Walk inward normally. At the first broken mirror pair, spend the one deletion token on either the left character or the right character.

**CORE MOVE**

Use two pointers from both ends. If characters match, keep moving inward. On the first mismatch, check whether skipping left or skipping right leaves a normal palindrome.

**CODE SKELETON**

```
left = 0; right = n - 1
while left < right:
    if s[left] == s[right]: left++; right--
    else:
        return isPalindrome(left + 1, right)
        || isPalindrome(left, right - 1)
return true
```

**BRUTE FORCE / BASELINE**

Try deleting every character once and test whether the result is a palindrome. It is easy to reason about but costs  $O(n^2)$  time if each deletion builds or checks a string.

**ALTERNATE APPROACHES**

- Recursive validation with a remaining-delete count works, but it is more general than this problem needs.
- Build two candidate substrings after the mismatch and compare them; simpler to see, but it uses extra string space.

**BUG MAGNETS**

- Branch only at the first mismatch; after spending the deletion, the rest must be a perfect palindrome.
- Try both skip-left and skip-right because either side may be the extra character.
- At most one deletion means already-palindromic strings are also valid.

## #11 ShortestUnsortedContinuousSubarray

Medium

Boundary violation scan Medium [Source](#) [LeetCode](#)

**Before reading: which scan gives the right boundary, and which scan gives the left boundary?**

**PROBLEM DESCRIPTION**

Given an integer array, return the length of the shortest continuous subarray that must be sorted so the whole array becomes non-decreasing.

**INPUT**

nums = [2, 6, 4, 8, 10, 9, 15]

**OUTPUT**

5

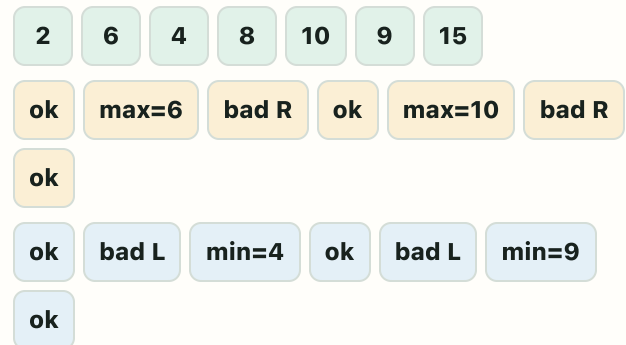
**TIME**  
**O(n)**

**SPACE**  
**O(1)**

**DATA STRUCTURES**  
Forward max scan, backward  
min scan, left/right boundaries

**WHEN TO RECOGNIZE IT**

When only one continuous middle window is broken, and values outside it should already fit sorted order.

**VISUAL HOOK**

- 1 From the left, track maxSoFar; smaller values prove the right edge must extend there.
- 2 From the right, track minSoFar; larger values prove the left edge must extend there.
- 3 The answer is right-left+1, or 0 if no boundary moved.

A value smaller than the best max seen from the left must be inside the window. A value larger than the best min seen from the right must also be inside it.

**CORE MOVE**

Scan left to right and push the right boundary whenever `nums[i]` is less than `maxSoFar`. Scan right to left and push the left boundary whenever `nums[i]` is greater than `minSoFar`.

**CODE SKELETON**

```
right = 0; maxSoFar = nums[0]
for i from 1 to n-1:
    if nums[i] < maxSoFar: right = i
    maxSoFar = max(maxSoFar, nums[i])
left = 0; minSoFar = nums[n-1]
for i from n-2 down to 0:
    if nums[i] > minSoFar: left = i
    minSoFar = min(minSoFar, nums[i])
return no violation ? 0 : right - left + 1
```

**BRUTE FORCE / BASELINE**

Copy and sort the array, then compare positions with the original. The first and last differences give the window, but sorting costs  $O(n \log n)$ .

**ALTERNATE APPROACHES**

- Use a monotonic stack to find the same left and right boundaries; it is useful practice but uses  $O(n)$  space.
- Find the initially decreasing window, then expand it by the min and max inside that window. This is also  $O(n)$ , but has more boundary cases.

**BUG MAGNETS**

- The forward scan finds the right boundary, not the left one.
- The backward scan finds the left boundary, not the right one.
- If no violation is found, return 0 for an already sorted array.

## #12 ContainerWithMostWater

Medium

Move the shorter wall Medium [Source](#) [LeetCode](#)**Before reading: why is the shorter wall the only pointer worth moving?****PROBLEM DESCRIPTION**

Given vertical line heights, choose two lines that form a container with the x-axis and return the maximum water area they can hold.

**INPUT**

height = [1, 8, 6, 2, 5, 4, 8, 3, 7]

**OUTPUT**

49

**TIME**  
 **$O(n)$** **SPACE**  
 **$O(1)$** **DATA STRUCTURES**  
**Two boundary pointers, max area**

**WHEN TO RECOGNIZE IT**

When the score depends on two endpoints, and width shrinks every time you move inward.

**VISUAL HOOK**

**area = min(left, right) \* width**    **move shorter side**

- 1 Compute area from the current left and right walls.
- 2 The shorter wall limits the water height.
- 3 Move the shorter wall inward to search for a taller cap before width shrinks too much.

The water height is capped by the shorter wall. Moving the taller wall inward only loses width while keeping the same short cap.

**CORE MOVE**

Start at both ends, compute area, update the best answer, then move the pointer at the shorter height inward.

**CODE SKELETON**

```
left = 0; right = n - 1; best = 0
while left < right:
    area = min(height[left], height[right]) *
    (right - left)
    best = max(best, area)
    if height[left] <= height[right]: left++
    else: right--
return best
```

**BRUTE FORCE / BASELINE**

Check every pair of lines and compute its area. This is obviously correct but takes  $O(n^2)$  time.

**ALTERNATE APPROACHES**

- There is no need to sort because positions define width; sorting would destroy the distance information.
- A monotonic structure is unnecessary here because each step safely discards the shorter boundary.

**BUG MAGNETS**

- Area uses the smaller height, not the larger one.
- Width is right-left, not the number of bars between them.
- Move the shorter wall; moving the taller wall cannot improve the current height limit.

## #13 BoatsToSavePeople

**Medium**

Pair lightest with heaviest    Medium    [Source](#)    [LeetCode](#)

**Before reading: why does the heaviest remaining person get decided immediately?**

**PROBLEM DESCRIPTION**

Given people weights and a boat limit, return the minimum number of boats needed when each boat can carry at most two people and total weight cannot exceed the limit.

**INPUT**

people = [3, 2, 2, 1]  
limit = 3

**OUTPUT**

3

**TIME**

$O(n \log n)$

**SPACE**

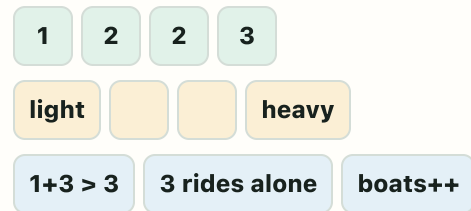
$O(\log n)$

**DATA STRUCTURES**

Sorted array, light/heavy pointers, boat counter

**WHEN TO RECOGNIZE IT**

When each trip can take at most two people, and the heaviest remaining person must be placed in the next boat.

**VISUAL HOOK**

- 1 Sort people so the lightest and heaviest remaining are visible.
- 2 The heaviest must use a boat now; check if the lightest can share it.
- 3 Count one boat every loop, and move the heavy pointer every loop.

Always decide the fate of the heaviest remaining person. If the lightest can fit with them, pair them; otherwise the heaviest must go alone.

**CORE MOVE**

Sort weights. Keep left at the lightest and right at the heaviest. Each loop uses one boat for right; move left too only when left+right fits.

**CODE SKELETON**

```
sort(people)
left = 0; right = n - 1; boats = 0
while left <= right:
    if people[left] + people[right] <= limit:
        left++
    right--
    boats++
return boats
```



**BRUTE FORCE / BASELINE**

Try all possible pairings and choose the fewest boats. This quickly becomes messy and expensive because each pairing choice affects the remaining people.

**ALTERNATE APPROACHES**

- Counting buckets can replace sorting if weights are tightly bounded, but sorting is simpler and general.
- A priority queue is possible, but it adds overhead and does not beat the sorted two-pointer reasoning.

**BUG MAGNETS**

- The heaviest person always consumes one boat in the current step.
- Only move the light pointer when the lightest person actually pairs with the heaviest.
- The loop must allow `left == right` because one remaining person still needs a boat.

## #14 SortArrayByParity

Easy

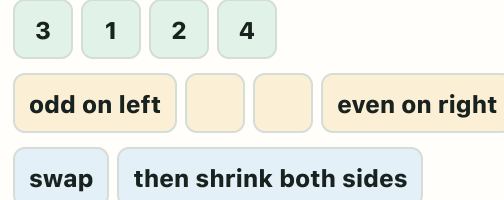
Parity partition Easy [Source](#) [LeetCode](#)**Before reading: what exact pair deserves a swap?****PROBLEM DESCRIPTION**

Given an integer array, move all even numbers before all odd numbers. Any valid order inside the even and odd groups is allowed.

**INPUT**`nums = [3, 1, 2, 4]`**OUTPUT**`[4, 2, 1, 3]`**TIME** **$O(n)$** **SPACE** **$O(1)$** **DATA STRUCTURES****Left/right pointers, in-place swaps**

**WHEN TO RECOGNIZE IT**

When the array only needs to be partitioned into two groups, and internal order does not matter.

**VISUAL HOOK**

- 1 Left pointer wants an even value.
- 2 Right pointer wants an odd value.
- 3 Swap only when left is odd and right is even; otherwise move the already-correct side.

Treat the left side as the even zone and the right side as the odd zone. Swap only when both ends are on the wrong side.

**CORE MOVE**

Move left past evens and right past odds. When left is odd and right is even, swap them and move both pointers inward.

**CODE SKELETON**

```
left = 0; right = n - 1
while left < right:
    if nums[left] is odd and nums[right] is even:
        swap(left, right); left++; right--
    else:
        if nums[left] is even: left++
        if nums[right] is odd: right--
return nums
```

**BRUTE FORCE / BASELINE**

Create a new array, first copy all evens, then copy all odds. It is simple but uses  $O(n)$  extra space.

**ALTERNATE APPROACHES**

- Use a write pointer to place evens at the front in one pass; this is also  $O(n)$  but may perform more swaps.
- If stable order were required, use an auxiliary array because this in-place partition does not preserve order.

**BUG MAGNETS**

- Any even-before-odd ordering is valid; do not preserve original order unless the problem asks.
- Move pointers over already-correct values before swapping.
- After a successful swap, both ends are fixed and both pointers can move.

## #15 Remove Duplicates From Sorted Array

Easy

Write unique values Easy [Source](#) [LeetCode](#)

**Before reading: where does the next new value get written?****PROBLEM DESCRIPTION**

Given a sorted array, remove duplicates in place so each value appears once at the front, and return the number of unique values.

**INPUT**

nums = [0, 0, 1, 1, 1, 2, 2, 3, 3, 4]

**OUTPUT**

5

**TIME**

**$O(n)$**

**SPACE**

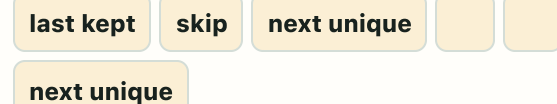
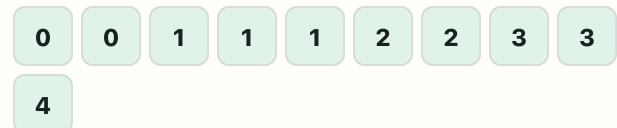
**$O(1)$**

**DATA STRUCTURES**

**Write boundary, scan pointer, sorted array**

**WHEN TO RECOGNIZE IT**

When the array is already sorted and duplicates appear in contiguous blocks.

**VISUAL HOOK**

**write at l+1 then advance**

- 1 Keep l on the last unique value already placed.
- 2 Move r until it finds a value different from nums[l].
- 3 Copy nums[r] to l+1, then advance the unique boundary.

Because equal values are adjacent, the scan pointer can jump over a duplicate block and copy the next new value right after the last kept value.

**CORE MOVE**

Keep l at the last unique value written. Move r across duplicates. When r finds a new value, write it to l+1 and advance both sides.

**CODE SKELETON**

```
l = 0; r = 1
while r < n:
    while r < n and nums[r] == nums[l]: r++
    if r == n: break
    nums[l + 1] = nums[r]
    l++; r++
return l + 1
```

**BRUTE FORCE / BASELINE**

Use a set or new list to collect unique values, then copy them back. It is easier but uses extra space and ignores the sorted in-place advantage.

**ALTERNATE APPROACHES**

- Use a write pointer starting at 1 and copy `nums[read]` whenever it differs from `nums[write-1]`.
- For the variant allowing at most two duplicates, compare against `nums[write-2]` instead.

**BUG MAGNETS**

- Return the count of unique values, not the full array.
- Only the first returned-length positions need to be correct.
- The sorted property is what makes adjacent duplicate skipping valid.

## #16 TwoSumSorted

Medium

Sorted sum squeeze Medium [Source](#) [LeetCode](#)

**Before reading: which pointer move makes the current sum move toward target?**

**PROBLEM DESCRIPTION**

Given a 1-indexed sorted array, return the two positions whose values add up to the target. Exactly one valid answer is expected.

**INPUT**

`numbers = [2, 7, 11, 15]`  
`target = 9`

**OUTPUT**

`[1, 2]`

**TIME**

**$O(n)$**

**SPACE**

**$O(1)$**

**DATA STRUCTURES**

**Left/right pointers, sorted array**

**WHEN TO RECOGNIZE IT**

When the array is sorted and the question asks for two values that sum to a target.

**VISUAL HOOK**

sum too big -> R--

sum too small -> L++

- 1 Left and right start at the smallest and largest remaining values.
- 2 If the sum is too small, increase it by moving left rightward.
- 3 If the sum is too large, decrease it by moving right leftward.

The sorted order gives direction. A sum that is too large can only improve by dropping the right pointer; a sum that is too small can only improve by raising the left pointer.

**CORE MOVE**

Start at both ends. Compare current sum to target. Move left up when the sum is too small, move right down when the sum is too large, and return 1-based indexes on match.

**CODE SKELETON**

```
left = 0; right = n - 1
while left < right:
    sum = numbers[left] + numbers[right]
    if sum == target: return [left + 1, right + 1]
    if sum < target: left++
    else: right--
```

**BRUTE FORCE / BASELINE**

Check every pair until the target sum appears. It is simple but takes  $O(n^2)$  time.

**ALTERNATE APPROACHES**

- Use a HashMap like classic Two Sum, but that ignores the sorted property and uses  $O(n)$  space.
- For each value, binary search for target-value; this is  $O(n \log n)$ , slower than two pointers.

**BUG MAGNETS**

- Return 1-based indexes, not Java's 0-based positions.
- Move only one pointer after comparing the sum.
- The two-pointer direction works because the input is sorted.

## #17 FindDuplicateNumber

Medium

Cycle entry in value graph Medium [Source](#) [LeetCode](#)

**Before reading: why can an array of values behave like a linked-list cycle?**

**PROBLEM DESCRIPTION**

Given  $n+1$  numbers where every value is in the range  $1..n$ , return the one repeated number without modifying the array and using only constant extra space.

**INPUT**

nums = [1, 3, 4, 2, 2]

**OUTPUT**

2

**TIME**  
 **$O(n)$**

**SPACE**  
 **$O(1)$**

**DATA STRUCTURES**  
Slow pointer, fast pointer,  
value-as-next-index graph

**WHEN TO RECOGNIZE IT**

When values are valid indexes and the constraints forbid sorting, marking, or extra storage.

**VISUAL HOOK**

0 1 3 2 4 2

jump by nums[i]    2 -> 4 -> 2

cycle entry = duplicate value

- 1 Treat nums[i] as the next index to jump to.
- 2 Use slow and fast jumps until they meet inside the loop.
- 3 Reset one pointer to 0; the second meeting is the loop entry and duplicate.

Read each value as the next index to jump to. The repeated value has two incoming jumps, so the path enters a loop at the duplicate.

**CORE MOVE**

Use Floyd's cycle detection. First let slow and fast meet inside the cycle. Then reset slow to 0 and move both one step at a time; their meeting point is the duplicate value.

**CODE SKELETON**

```
slow = 0; fast = 0
repeat:
    slow = nums[slow]
    fast = nums[nums[fast]]
until slow == fast
slow = 0
while slow != fast:
    slow = nums[slow]
    fast = nums[fast]
return slow
```

**BRUTE FORCE / BASELINE**

Compare each number against every other number or sort the array first. Both are simpler, but brute force is  $O(n^2)$  and sorting modifies or copies the array.

**ALTERNATE APPROACHES**

- Use a HashSet to remember seen values, but that violates the constant-space constraint.
- Binary search on value counts can keep  $O(1)$  space without modifying the array, but it is  $O(n \log n)$  and less visual.

**BUG MAGNETS**

- Move by value: next index is `nums[current]`, not `current+1`.
- After the first meeting, reset one pointer to 0, not to `nums[0]` in this implementation shape.
- The duplicate is the cycle entry value, not merely the first slow/fast meeting point.

## #18 MaximumProductOfTwoElementsInArray

Easy

Track top two Easy [Source](#) [LeetCode](#)**Before reading: what happens to the old max when a new max appears?****PROBLEM DESCRIPTION**

Given an array of positive integers, choose two different elements and return the maximum value of  $(a-1) * (b-1)$ .

**INPUT**`nums = [3, 4, 5, 2]`**OUTPUT**`12`**TIME** **$O(n)$** **SPACE** **$O(1)$** **DATA STRUCTURES****Largest value, second largest value**

**WHEN TO RECOGNIZE IT**

When the final score only needs the largest two values, not their positions or all pairs.

**VISUAL HOOK**

3

4

5

2

max1=5

max2=4

(5-1) \* (4-1)

=

12

- 1 Keep max1 as the largest value seen.
- 2 Keep max2 as the second largest value seen.
- 3 When current becomes max1, demote old max1 into max2 before replacing it.

Carry two trophies while scanning: the biggest seen so far and the runner-up. Only those two can maximize the product.

**CORE MOVE**

Scan once. If the current value beats max1, demote max1 into max2 and replace max1. Otherwise, update max2 if the value is between them.

**CODE SKELETON**

```
max1 = -inf; max2 = -inf
for value in nums:
    if value >= max1:
        max2 = max1
        max1 = value
    else if value >= max2:
        max2 = value
return (max1 - 1) * (max2 - 1)
```

**BRUTE FORCE / BASELINE**

Try every pair and compute the product. This is easy to verify but takes  $O(n^2)$  time.

**ALTERNATE APPROACHES**

- Sort the array and use the last two values; simpler but  $O(n \log n)$ .
- Use a max-heap and pop two values; works, but uses extra structure for a one-scan problem.

**BUG MAGNETS**

- Use two different elements; max2 must come from a separate position.
- When a new max1 appears, move the old max1 into max2 first.
- Apply the -1 after choosing the two largest numbers.

## #19 RearrangeBySign

Medium

Alternate signs Medium [Source](#) [LeetCode](#)

**Before reading: which slots are reserved for positives and negatives?**



**PROBLEM DESCRIPTION**

Given an array with equal counts of positive and negative numbers, rearrange it so positives and negatives alternate, starting with a positive, while preserving relative order within each sign group.

**INPUT**

nums = [3, 1, -2, -5, 2, -4]

**OUTPUT**

[3, -2, 1, -5, 2, -4]

**TIME**  
 **$O(n)$**

**SPACE**  
 **$O(n)$**

**DATA STRUCTURES**  
Output array, positive scan pointer, negative scan pointer

**WHEN TO RECOGNIZE IT**

When the target positions are fixed by sign: even indexes need positives and odd indexes need negatives.

**VISUAL HOOK**

0:+

1:-

2:+

3:-

4:+

5:-

pos scan

neg scan

write positive

write negative

jump +2

- 1 Even output positions get the next positive number.
- 2 Odd output positions get the next negative number.
- 3 Move both source scanners forward only after their next matching value is used.

Think of the result as alternating slots. Fill each even slot with the next positive and the following odd slot with the next negative.

**CORE MOVE**

Keep one pointer searching for the next positive and one pointer searching for the next negative. Write them into result positions  $i$  and  $i+1$ , then move  $i$  by two.

**CODE SKELETON**

```
result = int[n]
pos = 0; neg = 0
for i = 0; i < n; i += 2:
    move pos to next positive
    move neg to next negative
    result[i] = nums[pos]
    result[i + 1] = nums[neg]
    pos++; neg++
return result
```

**BRUTE FORCE / BASELINE**

Build two separate lists, one for positives and one for negatives, then merge them alternately. It is clear but still uses  $O(n)$  extra space.

**ALTERNATE APPROACHES**

- Use direct write indexes: positiveIndex = 0 and negativeIndex = 1, then place each scanned value in the matching slot.
- If relative order did not matter, in-place partitioning could be considered, but this problem expects stable sign order.

**BUG MAGNETS**

- The output starts with a positive at index 0.
- Preserve relative order within positives and within negatives.
- Advance the result index by two after writing one positive-negative pair.